Test Plan

Brew & Bean Anomaly Detection Dashboard

1. Purpose and scope

This plan describes how the Brew & Bean anomaly detection project is tested: the data cleaning module (clean_data.py), the prediction step (predict.py with the IsolationForest model), and the Streamlit dashboard (app.py). It covers unit, integration, UI (functional) and performance tests. All data in the project is synthetic.

2. Test levels at a glance

Level	What is tested	How	When
Unit	clean() and engineer() in clean_data.py	Automated, pytest (tests/test_clean_data.py)	Every push, via GitHub Actions
Integration	clean + engineer + model together (predict.py), model file, evaluation	Run scripts and check output	Before each release
UI / functional	Filters, table, chart, download button in app.py	Manual checklist in the browser	Before each release
Performance	Load and response time of the dashboard	Manual timing	Before each release

3. Unit tests (automated)

Run with `python -m pytest`. A small fake table with deliberate missing values is used, so the expected answers are known in advance.

ID	Test	Expected result
U1	Missing amount is filled	No empty amounts; the filled value is the median (10 and 30 give 20.0)
U2	Missing item and payment_method are filled	Empty values become "unknown"
U3	Row with an invalid date is dropped	3 rows with 1 bad date leaves 2 rows
U4	Time features	hour_of_day is 8, 14, 23 and day_of_week is 3, 4, 5 for the sample dates
U5	log_amount	Equals log(1 + amount)
U6	One-hot payment columns	payment_method_Cash, _Card and _unknown columns exist; a Cash row has 1 in the Cash column

4. Integration tests

ID	Test	Steps	Expected result
I1	Prediction pipeline	Run `python predict.py`	Output has an anomaly column (True/False) and a score column; the share flagged is close to the model contamination value
I2	Model file loads	joblib.load("model/isolation_forest.pkl")	Loads without error and prints an IsolationForest
I3	Model quality	Run `python evaluation.py`	ROC AUC is at least 0.80; confusion_matrix.png is created
I4	App reads data and predicts	Start the app and open it	Table and chart appear with no error box and no traceback in the terminal

5. UI and functional tests (manual)

Run `streamlit run app.py` (or open the live URL) and check each item.

ID	Test	Steps	Expected result
F1	Date filter	Pick a short date range in the sidebar	Row count above the table goes down; all dates are inside the range
F2	Hour filter	Move the Hour of Day slider, e.g. 6 to 10	Only rows in those hours are shown
F3	Anomaly column	Look at the table	A column anomaly shows True/False; True rows are highlighted red
F4	Chart	Look at the Amount over time chart	Blue line for amounts; red dots only at anomaly points
F5	Download	Click the download button and open the CSV	File flagged_transactions.csv contains only anomaly = True rows
F6	Empty result	Choose a filter that matches no rows	A warning message is shown; the app does not crash
F7	One date only	Click only one date in the date picker	An info message asks for both dates; no error
F8	Public access	Open the live URL in a private/incognito window	App opens without any login

6. Performance tests (manual targets)

ID	Test	How to measure	Target
P1	First page load	Time from opening the URL to the table appearing (stopwatch)	Under 10 seconds (the first visit after the free app has been asleep can take longer)
P2	Filter response	Change a filter and time until the table updates	Under 3 seconds
P3	Prediction speed	Time predict_anomalies on about 5000 rows (time.time() before and after)	Under 5 seconds
P4	Download	Click the download button	File is ready in under 2 seconds

7. Test environment

Where	Details
Local	Windows, Python 3.13, VS Code terminal, packages from requirements.txt
GitHub Actions	ubuntu-latest, Python 3.11, runs python -m pytest on every push and pull request
Live app	Streamlit Community Cloud (free), deployed from the main branch, app file app.py

8. Pass and fail rules

A release is accepted when all unit tests pass in GitHub Actions (green CI badge), integration tests I1 to I4 give the expected result, UI tests F1 to F8 pass, and the performance targets are met. Any failed item is fixed and the affected tests are run again before the next release.